ANGULAR CYBERPUNK

HACKER TERMINÁL & TŰZFAL VEZÉRLŐ

A projekt célja és a sztori: Ebben a haladó feladatban egy elit kiber-hálózat biztonsági mérnöke vagy. A feladatod egy adatbázis-figyelő és tűzfal-vezérlő terminál elkészítése. A szerveren tárolt adatcsomagok (pl. *titkos_tervrajz*, *jelszo_lista*) egy része titkosítva van az illetéktelenek elől, és a veszélyességi szintjüket is egyedi formátumban kell megjeleníteni. Az adatcsomagok ráadásul reagálnak a vizsgálatra: ha túl sokáig elemzed őket, riasztást váltanak ki, ha pedig rákattintasz egyre, karanténba zárod!

Ebben a feladatban a strukturált adatkezelést (`constructor` vs. `ngOnInit`), két saját Pipe-ot, és két komplexebb attribútum Direktívát kell ötvöznöd.

1. Adatmodell és a Komponens Életciklusa

Készíts egy `HackerTerminalComponent`-et, amely tárolja és betölti az adatcsomagokat.

- **Adatmodell (Interface):** Hozz létre egy `AdatCsomag` interfészt a következő mezőkkel: `id` (szám), `fajlnev` (szöveg), `kockazat` (szám 1–100 között), és `vedett` (logikai).
- **A constructor feladata:** Itt kizárólag a függőségek beinjektálását végezd el (pl. `ChangeDetectorRef`), illetve helyezz el egy `console.log('1. Constructor: Terminál inicializálása...')` hívást. *Fontos: a konstruktorban még nem töltünk be üzleti adatokat.*
- **Az ngOnInit:** Az adatok betöltése itt történjen! Tölts fel egy tömböt legalább 5 különböző kiberpunk adatcsomaggal. Ezen felül az `ngOnInit`-ben indíts el egy időzítőt (`setInterval`), ami 2 másodpercenként kiírja a konzolra: `"Szkennelés folyamatban... Rendszer stabil."`. (Ne felejtsd el az `ngOnDestroy`-ban leállítani az időzítőt!)

2. A transzformációk

A HTML-ben a nyers adatokat tilos megjeleníteni, mindent át kell alakítani a pipe-okkal!

A) Kódoló (kodolo.pipe.ts)

- **Feladat:** Ha az adatcsomag `vedett` értéke `true`, a pipe rejtse el a fájlnevét az illetéktelenek elől.
- **Logika:** Ha `vedett === true`, cserélje le az összes számot és magánhangzót egy `#` szimbólumra, ÉS fűzze elé a `[TITKOS]` előtagot. Ha a `vedett` paraméter `false`, adja vissza a fájlnevet csupa nagybetűvel, és fűzze utána a `.exe` kiterjesztést (pl. `ADATBAZIS.EXE`).

B) Kockázat Formázó (kockazat.pipe.ts)

- **Feladat:** Az 1–100 közötti kockázati szintet vizuális, riasztási skálává alakítja.
- **Logika:**
  - Ha a kockázat < 35: `[Szint: XX] [ALACSONY] Alacsony kockázat`
  - Ha a kockázat 35 – 74: `[Szint: XX] [GYANÚS] Gyanús forgalom`
  - Ha a kockázat >= 75: `[Szint: XX] [KRITIKUS] !!! KRITIKUS VÍRUS !!!`

3. A Két Saját Direktíva (Interakciók)

Az adatcsomagok (pl. egy `<li>` vagy `<div>` kártya) reagáljanak az egérre az alábbi két direktívával!

A) Behatolás Érzékelő Direktíva (behatolasErzekelo.directive.ts)

- **Feladat:** Ha a felhasználó túl sokáig vizsgál (viszi rá az egeret) egy adatcsomagot, a tűzfal riasztást ad le.
- **Logika:**
  - A `@HostListener('mouseenter')` eseményben indíts el egy `setTimeout`-ot (1.5 másodperc).
  - Ha az 1.5 másodperc letelik (és az egeret még nem húzták el), a `@HostBinding` (vagy `Renderer2`) segítségével adj az elemhez egy `riado` CSS osztályt (ami pl. vörösen villog és megemeli a kártyát).
  - A `@HostListener('mouseleave')` eseményben töröld az időzítőt (`clearTimeout`), hogy ha hamarabb elhúzza az egeret, ne kapjon riasztást, illetve ha már megkapta, vedd le róla a CSS osztályt.

B) Karantén Direktíva (karanten.directive.ts)

- **Feladat:** Kezeli a kattintást, amivel az adatcsomagot karanténba zárjuk.
- **Logika:**
  - Figyeld a `@HostListener('click')` eseményt.
  - Kattintáskor az elem kapjon egy `karantenban` CSS osztályt, ami (SCSS `transition` segítségével) elhalványítja (`opacity: 0.5`), áthúzza a szöveget, és ciánkék neonszegélyt ad neki.
  - **Kihívás:** A direktíva egy `@Output()` eseménykibocsátón keresztül jelezzen vissza a Főkomponensnek, hogy a zárolás megtörtént, ami erre reagálva növeljen egy számlálót és írja ki a képernyőre: `"Fájl karanténba helyezve! Összesen zárolva: X db"`.

4. Összeszerelés (A Felület)

A `HackerTerminalComponent` HTML sablonjában jelenítsd meg az adatcsomagok listáját kártyák formájában.

Minden kártyán alkalmazd:

1. A **Kódoló Pipe**-ot a fájlnévre (kösd be paraméterként az objektum `vedett` tulajdonságát).
2. A **Kockázat Pipe**-ot a számértékre.
3. A **Behatolás Érzékelő** és a **Karantén** direktívákat magára a kártyát befoglaló HTML elemre.

Tervezz hozzá egy sötét, mátrix/terminál stílusú SCSS dizájnt (fekete háttér, neonzöld és ciánkék betűk), ami életre kelti a direktívák által hozzáadott osztályokat (`.riado`, `.karantenban`)!

Segédlet: Kezdő tömb és animáció minta

```
this.adatcsomagok = [
  { id: 1, fajlnev: 'rendszermag_kulcs_01', kockazat: 88, vedett: true },
  { id: 2, fajlnev: 'felhasznaloi_naplo', kockazat: 20, vedett: false },
  { id: 3, fajlnev: 'tuzfal_konfig_2026', kockazat: 54, vedett: true },
  { id: 4, fajlnev: 'trojai_betolto', kockazat: 95, vedett: false },
  { id: 5, fajlnev: 'titkos_tervrajz_9', kockazat: 67, vedett: true },
  { id: 6, fajlnev: 'halozati_ping', kockazat: 12, vedett: false }
];
```

```
/* Riadó villogás animáció: */
@keyframes riadoVillogas {
  0%   { box-shadow: 0 0 5px #ff003c; transform: translateY(0); }
  50%  { box-shadow: 0 0 20px #ff003c; transform: translateY(-3px); }
  100% { box-shadow: 0 0 5px #ff003c; transform: translateY(0); }
}

.riado {
  border-color: #ff003c !important;
  animation: riadoVillogas 0.4s infinite !important;
}

/* Karantén stílus: */
.karantenban {
  opacity: 0.5;
  text-decoration: line-through;
  border-color: #00f3ff !important;
  box-shadow: 0 0 12px rgba(0, 243, 255, 0.5);
}
```